Finding Bugs and Features Using Cryptographically-Informed Functional Testing

Fernando Virdia
University of Surrey

Joint work with Giacomo Fenzi (EPFL Lausanne) and Jan Gilcher (ETH Zurich)

Real World Crypto 2026

During ASIACRYPT 2017 in Hong Kong, I was asked:

During ASIACRYPT 2017 in Hong Kong, I was asked:

"Suppose you implement a cryptographic hash function H.
How do you know you got it right?"

During ASIACRYPT 2017 in Hong Kong, I was asked:

"Suppose you implement a cryptographic hash function H.
How do you know you got it right?"

- Obviously, you may want to test your implementation

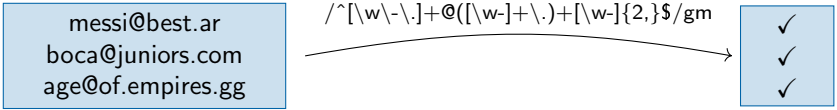
Testing “predictable” functions

Testing “predictable” functions

- Suppose you ~~plagiarised~~ wrote a regex for validating email addresses
- Writing your own test suite is relatively easy

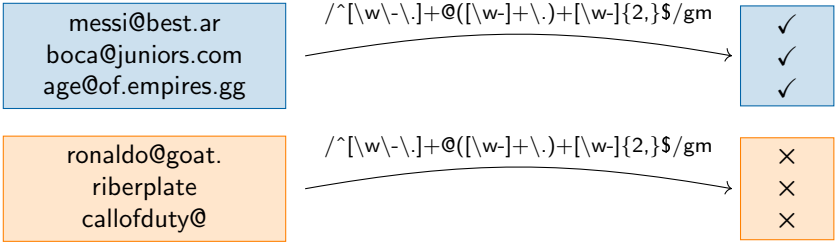
Testing “predictable” functions

- Suppose you ~~plagiarised~~ wrote a regex for validating email addresses
- Writing your own test suite is relatively easy



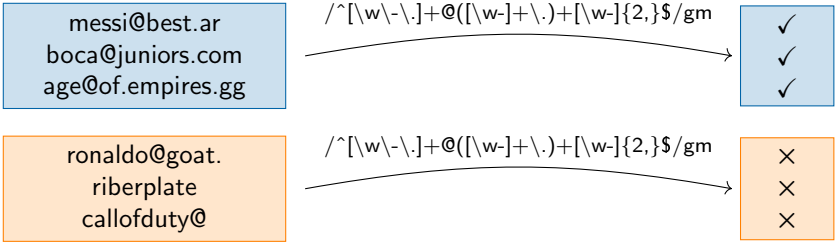
Testing "predictable" functions

- Suppose you ~~plagiarised~~ wrote a regex for validating email addresses
- Writing your own test suite is relatively easy



Testing “predictable” functions

- Suppose you ~~plagiarised~~ wrote a regex for validating email addresses
- Writing your own test suite is relatively easy



- However, we are not testing “predictable” functions

Testing cryptographic functions

Testing cryptographic functions

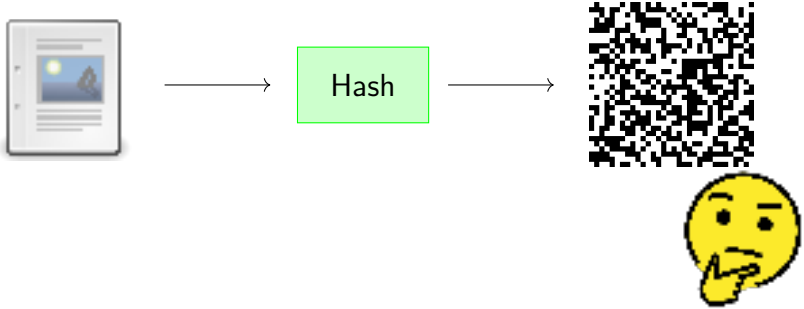
Catch

Cryptographic SW's output is often **pseudorandom**.

Testing cryptographic functions

Catch

Cryptographic SW's output is often **pseudorandom**.



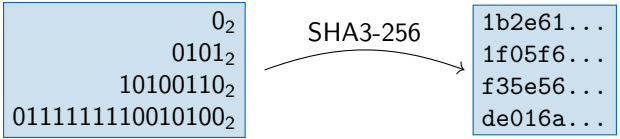
CC BY-SA 4.0 OpenMoji.org

Testing cryptographic functions

- For cryptographic SW, the traditional solution is to use Known Answer Tests (KATs)

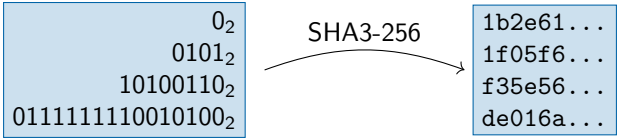
Testing cryptographic functions

- For cryptographic SW, the traditional solution is to use Known Answer Tests (KATs)



Testing cryptographic functions

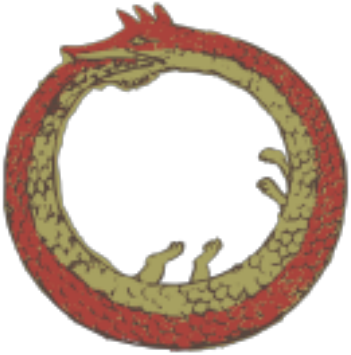
- For cryptographic SW, the traditional solution is to use Known Answer Tests (KATs)



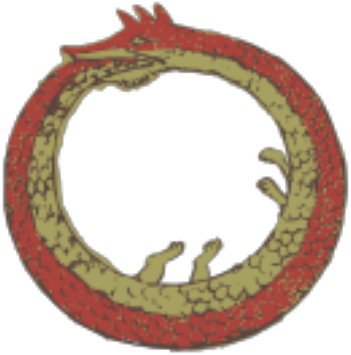
- You are checking against an existing implementation

- What if you were the designer of H? How do you test **before** KATs are available?

- What if you were the designer of H? How do you test **before** KATs are available?



- What if you were the designer of H? How do you test **before** KATs are available?



- Mouha *et al.* (IEEE Trans. on Reliability, 2018) observe:

- What if you were the designer of H? How do you test **before** KATs are available?

- Mouha *et al.* (IEEE Trans. on Reliability, 2018) observe: we don't know the output of H, but we know its cryptographic properties. — Use these properties to inform your testing!

Definition: collision resistance

Given $H : \{0,1\}^* \to \{0,1\}^n$, it's hard to find *distinct* $x, y \in \{0,1\}^*$ such that $H(x) = H(y)$.

Note: For hash functions is a cryptographic property *and a functional requirement*.

Definition: collision resistance

Given $H : \{0,1\}^* \to \{0,1\}^n$, it's hard to find *distinct* $x, y \in \{0,1\}^*$ such that $H(x) = H(y)$.

Note: For hash functions is a cryptographic property *and a functional requirement*.

Example: test collision resistance (bit-contribution)

Let $x \in \{0,1\}^4$ be a 4-bit string.

Definition: collision resistance

Given $H : \{0,1\}^* \to \{0,1\}^n$, it's hard to find *distinct* $x, y \in \{0,1\}^*$ such that $H(x) = H(y)$.

Note: For hash functions is a cryptographic property *and a functional requirement*.

Example: test collision resistance (bit-contribution)

Let $x \in \{0,1\}^4$ be a 4-bit string. **Note:** most architectures work with byte-buffers!

$$x = \boxed{1 \mid 1 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0}$$

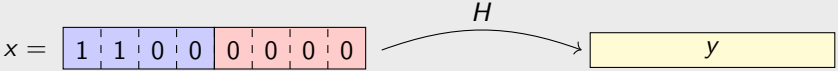
Definition: collision resistance

Given $H : \{0,1\}^* \to \{0,1\}^n$, it's hard to find *distinct* $x, y \in \{0,1\}^*$ such that $H(x) = H(y)$.

Note: For hash functions is a cryptographic property *and a functional requirement*.

Example: test collision resistance (bit-contribution)

Let $x \in \{0,1\}^4$ be a 4-bit string. **Note:** most architectures work with byte-buffers!



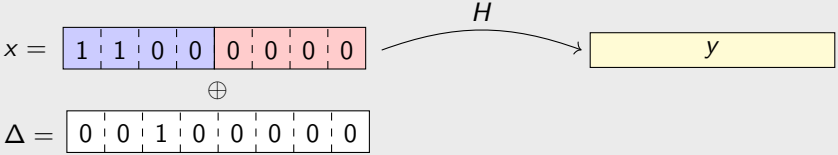
Definition: collision resistance

Given $H : \{0,1\}^* \to \{0,1\}^n$, it's hard to find *distinct* $x, y \in \{0,1\}^*$ such that $H(x) = H(y)$.

Note: For hash functions is a cryptographic property *and a functional requirement*.

Example: test collision resistance (bit-contribution)

Let $x \in \{0,1\}^4$ be a 4-bit string. **Note:** most architectures work with byte-buffers!



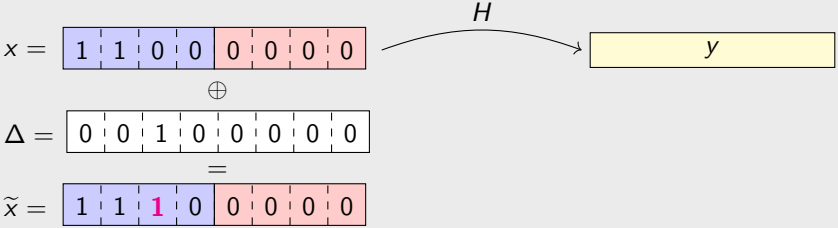
Definition: collision resistance

Given $H : \{0,1\}^* \to \{0,1\}^n$, it's hard to find *distinct* $x, y \in \{0,1\}^*$ such that $H(x) = H(y)$.

Note: For hash functions is a cryptographic property *and a functional requirement*.

Example: test collision resistance (bit-contribution)

Let $x \in \{0,1\}^4$ be a 4-bit string. **Note:** most architectures work with byte-buffers!



Definition: collision resistance

Given $H : \{0,1\}^* \to \{0,1\}^n$, it's hard to find *distinct* $x, y \in \{0,1\}^*$ such that $H(x) = H(y)$.

Note: For hash functions is a cryptographic property *and a functional requirement*.

Example: test collision resistance (bit-contribution)

Let $x \in \{0,1\}^4$ be a 4-bit string. **Note:** most architectures work with byte-buffers!

$$
\begin{array}{rl}
x = & \boxed{1\,1\,0\,0}\boxed{0\,0\,0\,0} \xrightarrow{H} \boxed{y} \\
 & \oplus \\
\Delta = & \boxed{0\,0\,1\,0\,0\,0\,0\,0} \\
 & = \\
\widetilde{x} = & \boxed{1\,1\,\mathbf{1}\,0}\boxed{0\,0\,0\,0} \xrightarrow{H} \boxed{\widetilde{y}}
\end{array}
$$

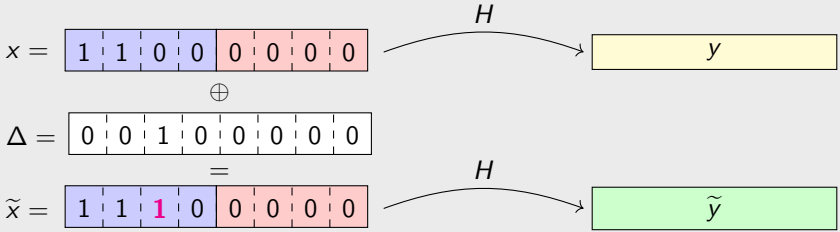
Definition: collision resistance

Given $H : \{0,1\}^* \to \{0,1\}^n$, it's hard to find *distinct* $x, y \in \{0,1\}^*$ such that $H(x) = H(y)$.

Note: For hash functions is a cryptographic property *and a functional requirement*.

Example: test collision resistance (bit-contribution)

Let $x \in \{0,1\}^4$ be a 4-bit string. **Note:** most architectures work with byte-buffers!



Check that $\tilde{y} \neq y$.

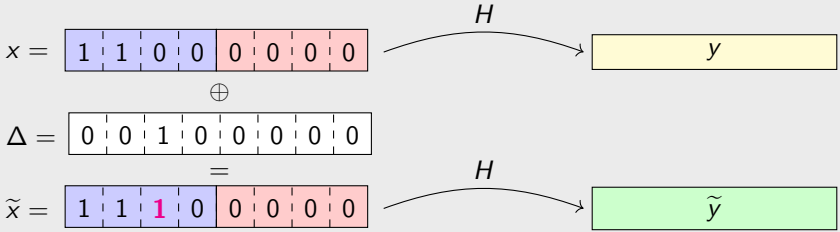
Definition: collision resistance

Given $H : \{0,1\}^* \to \{0,1\}^n$, it's hard to find *distinct* $x, y \in \{0,1\}^*$ such that $H(x) = H(y)$.

Note: For hash functions is a cryptographic property *and a functional requirement*.

Example: test collision resistance (bit-contribution)

Let $x \in \{0,1\}^4$ be a 4-bit string. **Note:** most architectures work with byte-buffers!



Check that $\tilde{y} \neq y$. Δ affects **output-contributing bits** of x, should result in different digests.

Definition: determinism

Given $H : \{0,1\}^* \to \{0,1\}^n$, and $x \in \{0,1\}^*$, $H(x)$ is a fixed value.

Definition: determinism

Given $H : \{0,1\}^* \to \{0,1\}^n$, and $x \in \{0,1\}^*$, $H(x)$ is a fixed value.

Example: test determinism *under neighboring memory* changes (bit-exclusion)

Let $x \in \{0,1\}^4$ be a 4-bit string.

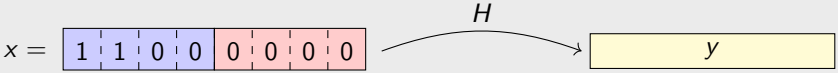
$$x = \boxed{1 \mid 1 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0}$$

Definition: determinism

Given $H : \{0,1\}^* \to \{0,1\}^n$, and $x \in \{0,1\}^*$, $H(x)$ is a fixed value.

Example: test determinism *under neighboring memory* changes (bit-exclusion)

Let $x \in \{0,1\}^4$ be a 4-bit string.



Definition: determinism

Given $H : \{0,1\}^* \to \{0,1\}^n$, and $x \in \{0,1\}^*$, $H(x)$ is a fixed value.

Example: test determinism *under neighboring memory* changes (bit-exclusion)

Let $x \in \{0,1\}^4$ be a 4-bit string.

$$
\begin{array}{rcl}
x = & \boxed{1\,1\,0\,0\,0\,0\,0\,0} & \xrightarrow{H} \boxed{y} \\
 & \oplus & \\
\Delta = & \boxed{0\,0\,0\,0\,0\,0\,1\,0} & \\
 & = & \\
\widetilde{x} = & \boxed{1\,1\,0\,0\,0\,0\,\mathbf{1}\,0} &
\end{array}
$$

Definition: determinism

Given $H : \{0,1\}^* \to \{0,1\}^n$, and $x \in \{0,1\}^*$, $H(x)$ is a fixed value.

Example: test determinism *under neighboring memory* changes (bit-exclusion)

Let $x \in \{0,1\}^4$ be a 4-bit string.

$$
\begin{array}{rcl}
x = & \boxed{1 \mid 1 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0} & \xrightarrow{H} \boxed{y} \\
 & \oplus & \\
\Delta = & \boxed{0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 1 \mid 0} & \\
 & = & \\
\widetilde{x} = & \boxed{1 \mid 1 \mid 0 \mid 0 \mid 0 \mid 0 \mid \mathbf{1} \mid 0} & \xrightarrow{H} \boxed{\widetilde{y}}
\end{array}
$$

Definition: determinism

Given $H : \{0,1\}^* \to \{0,1\}^n$, and $x \in \{0,1\}^*$, $H(x)$ is a fixed value.

Example: test determinism *under neighboring memory* changes (bit-exclusion)

Let $x \in \{0,1\}^4$ be a 4-bit string.

$$
\begin{array}{rcl}
x = & \boxed{1 \mid 1 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0} & \xrightarrow{H} \boxed{y} \\
 & \oplus & \\
\Delta = & \boxed{0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 1 \mid 0} & \\
 & = & \\
\widetilde{x} = & \boxed{1 \mid 1 \mid 0 \mid 0 \mid 0 \mid 0 \mid \mathbf{1} \mid 0} & \xrightarrow{H} \boxed{\widetilde{y}}
\end{array}
$$

Check that $\widetilde{y} = y$.

Definition: determinism

Given $H : \{0,1\}^* \to \{0,1\}^n$, and $x \in \{0,1\}^*$, $H(x)$ is a fixed value.

Example: test determinism *under neighboring memory* changes (bit-exclusion)

Let $x \in \{0,1\}^4$ be a 4-bit string.

$$
\begin{array}{rcl}
x = & \boxed{1 \mid 1 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0} & \xrightarrow{H} \boxed{y} \\
 & \oplus & \\
\Delta = & \boxed{0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 1 \mid 0} & \\
 & = & \\
\widetilde{x} = & \boxed{1 \mid 1 \mid 0 \mid 0 \mid 0 \mid 0 \mid \mathbf{1} \mid 0} & \xrightarrow{H} \boxed{\widetilde{y}}
\end{array}
$$

Check that $\widetilde{y} = y$.

- If Δ does not affect output-contributing bits, outputs should not change

Definition: determinism

Given $H : \{0,1\}^* \to \{0,1\}^n$, and $x \in \{0,1\}^*$, $H(x)$ is a fixed value.

Example: test determinism *under neighboring memory* changes (bit-exclusion)

Let $x \in \{0,1\}^4$ be a 4-bit string.

$$
\begin{array}{rcl}
x = & \boxed{1\,1\,0\,0\,0\,0\,0\,0} & \xrightarrow{H} \boxed{y} \\
 & \oplus & \\
\Delta = & \boxed{0\,0\,0\,0\,0\,0\,1\,0} & \\
 & = & \\
\widetilde{x} = & \boxed{1\,1\,0\,0\,0\,0\,\mathbf{1}\,0} & \xrightarrow{H} \boxed{\widetilde{y}}
\end{array}
$$

Check that $\widetilde{y} = y$.

- If Δ does not affect output-contributing bits, outputs should not change
- If $y \neq \widetilde{y}$, likely a buffer over-read!

- Is this idea effective? Mouha *et al.*: for hashes, yes!

- Is this idea effective? Mouha *et al.*: for hashes, yes!
- In our work, we explore this idea for KEMs and DSSs.

- Is this idea effective? Mouha *et al.*: for hashes, yes!
- In our work, we explore this idea for KEMs and DSSs.

Our contributions

- We wrote cryptographically-informed test suites for KEMs and DSSs.
- We tested hundreds of implementations from NIST PQC.
- We found many bugs and *features*.

- Is this idea effective? Mouha *et al.*: for hashes, yes!
- In our work, we explore this idea for KEMs and DSSs.

Our contributions

- We wrote cryptographically-informed test suites for KEMs and DSSs.
- We tested hundreds of implementations from NIST PQC.
- We found many bugs and *features*.

- In this talk: KEMs. For DSSs, see the paper.

A refresher on KEMs

A refresher on KEMs

Mathematically speaking, a KEM consists of 3 functions:

$$\left.\begin{array}{l}\text{Key}\\ \text{Encapsulation}\\ \text{Mechanism}\end{array}\right\}$$

A refresher on KEMs

Mathematically speaking, a KEM consists of 3 functions:

Key
Encapsulation
Mechanism

$$\boxed{\text{Keygen}} : \$ \mapsto (\mathsf{sk}, \mathsf{pk})$$

A refresher on KEMs

Mathematically speaking, a KEM consists of 3 functions:

$$\text{Key Encapsulation Mechanism} \begin{cases} \boxed{\text{Keygen}} : \$ \mapsto (\mathsf{sk}, \mathsf{pk}) \\ \boxed{\text{Encaps}} : (\mathsf{pk}, \$) \mapsto (\mathsf{c}, \mathsf{k}) \end{cases}$$

A refresher on KEMs

Mathematically speaking, a KEM consists of 3 functions:

$$\text{Key Encapsulation Mechanism} \begin{cases} \boxed{\text{Keygen}} : \$ \mapsto (\mathsf{sk}, \mathsf{pk}) \\ \boxed{\text{Encaps}} : (\mathsf{pk}, \$) \mapsto (\mathsf{c}, \mathsf{k}) \\ \boxed{\text{Decaps}} : (\mathsf{sk}, \mathsf{c}) \mapsto \mathsf{k} \end{cases}$$

A refresher on KEMs

Mathematically speaking, a KEM consists of 3 functions:

$$\text{Key Encapsulation Mechanism} \begin{cases} \boxed{\text{Keygen}} : \$ \mapsto (\mathsf{sk}, \mathsf{pk}) \\ \boxed{\text{Encaps}} : (\mathsf{pk}, \$) \mapsto (\mathsf{c}, \mathsf{k}) \\ \boxed{\text{Decaps}} : (\mathsf{sk}, \mathsf{c}) \mapsto \mathsf{k} \end{cases}$$

IND-CCA2 Security

For a given $(\mathsf{sk}, \mathsf{pk})$ pair, challenge encapsulation c^* and key k^*, an adversary with access to a $\mathsf{Decaps}_{\mathsf{sk}}(\cdot)$ oracle cannot tell whether $\mathsf{Decaps}_{\mathsf{sk}}(\mathsf{c}^*) \stackrel{?}{=} \mathsf{k}^*$.

A refresher on KEMs

Mathematically speaking, a KEM consists of 3 functions:

$$\text{Key Encapsulation Mechanism} \begin{cases} \boxed{\text{Keygen}} : \$ \mapsto (\mathsf{sk}, \mathsf{pk}) \\ \boxed{\text{Encaps}} : (\mathsf{pk}, \$) \mapsto (\mathsf{c}, \mathsf{k}) \\ \boxed{\text{Decaps}} : (\mathsf{sk}, \mathsf{c}) \mapsto \mathsf{k} \end{cases}$$

IND-CCA2 Security

For a given $(\mathsf{sk}, \mathsf{pk})$ pair, challenge encapsulation c^* and key k^*, an adversary with access to a $\mathsf{Decaps}_{\mathsf{sk}}(\cdot)$ oracle cannot tell whether $\mathsf{Decaps}_{\mathsf{sk}}(\mathsf{c}^*) \stackrel{?}{=} \mathsf{k}^*$.

- Informally, ciphertext is non-malleable!

Using IND-CCA2 to define a functional test

Using IND-CCA2 to define a functional test

Example: test ciphertext non-malleability

Using IND-CCA2 to define a functional test

Example: test ciphertext non-malleability

$$\boxed{\text{Keygen}} \longrightarrow (\mathsf{sk}, \mathsf{pk})$$

Using IND-CCA2 to define a functional test

Example: test ciphertext non-malleability

$$\boxed{\text{Keygen}} \longrightarrow (\mathsf{sk}, \mathsf{pk})$$

$$\mathsf{pk} \longrightarrow \boxed{\text{Encaps}} \longrightarrow (\mathsf{c}, \mathsf{k})$$

Using IND-CCA2 to define a functional test

Example: test ciphertext non-malleability

$$\boxed{\text{Keygen}} \longrightarrow (\mathsf{sk}, \mathsf{pk})$$

$$\mathsf{pk} \longrightarrow \boxed{\text{Encaps}} \longrightarrow (\mathsf{c}, \mathsf{k})$$

$$\widetilde{\mathsf{c}} = \mathsf{c} \oplus \boxed{0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 1 \mid 0}$$

Using IND-CCA2 to define a functional test

Example: test ciphertext non-malleability

$$\boxed{\text{Keygen}} \longrightarrow (\mathsf{sk}, \mathsf{pk})$$

$$\mathsf{pk} \longrightarrow \boxed{\text{Encaps}} \longrightarrow (\mathsf{c}, \mathsf{k})$$

$$\widetilde{\mathsf{c}} = \mathsf{c} \oplus \boxed{0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 1 \mid 0}$$

$$(\mathsf{sk}, \widetilde{\mathsf{c}}) \longrightarrow \boxed{\text{Decaps}} \longrightarrow \widetilde{\mathsf{k}}$$

Using IND-CCA2 to define a functional test

Example: test ciphertext non-malleability

$$\boxed{\text{Keygen}} \longrightarrow (\mathsf{sk}, \mathsf{pk})$$

$$\mathsf{pk} \longrightarrow \boxed{\text{Encaps}} \longrightarrow (\mathsf{c}, \mathsf{k})$$

$$\widetilde{\mathsf{c}} = \mathsf{c} \oplus \boxed{0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 1 \mid 0}$$

$$(\mathsf{sk}, \widetilde{\mathsf{c}}) \longrightarrow \boxed{\text{Decaps}} \longrightarrow \widetilde{\mathsf{k}}$$

Check that $\mathsf{k} \neq \widetilde{\mathsf{k}}$.

Using IND-CCA2 to define a functional test

Example: test ciphertext non-malleability

$$\boxed{\text{Keygen}} \longrightarrow (\mathsf{sk}, \mathsf{pk})$$

$$\mathsf{pk} \longrightarrow \boxed{\text{Encaps}} \longrightarrow (\mathsf{c}, \mathsf{k})$$

$$\widetilde{\mathsf{c}} = \mathsf{c} \oplus \boxed{0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 1 \mid 0}$$

$$(\mathsf{sk}, \widetilde{\mathsf{c}}) \longrightarrow \boxed{\text{Decaps}} \longrightarrow \widetilde{\mathsf{k}}$$

Check that $\mathsf{k} \neq \widetilde{\mathsf{k}}$.

- Let's look at how we implemented this.

Decaps-Maul(c) Test (IND-CCA2 violations)

```
sus ← [ ];
Δ ← 1||0||...||0
(c, k, sk) ← GenInput()
repeat 8 · bytelen(c) times
    c̃, Δ′ ← Maul(c, Δ)
    try
        k̃ ← Decaps_sk(c̃)
        assert(k ≠ k̃)
    catch
        sus.append((c, Δ))
    Δ ← Δ′
return sus
```

GenInput()

```
(sk, pk) ← Keygen()
(c, k) ← Encaps_pk()
return (c, k, sk)
```

Maul(c, Δ)

```
c̃ ← c ⊕ Δ
Δ′ ← Δ ≫ 1
return (c̃, Δ′)
```

Decaps-Maul(c) Test (IND-CCA2 violations)

```
sus ← [ ];
Δ ← 1||0||...||0
(c, k, sk) ← GenInput()
repeat 8 · bytelen(c) times
    c̃, Δ′ ← Maul(c, Δ)
    try
        k̃ ← Decaps_sk(c̃)
        assert(k ≠ k̃)
    catch
        sus.append((c, Δ))
    Δ ← Δ′
return sus
```

GenInput()

```
(sk, pk) ← Keygen()
(c, k) ← Encaps_pk()
return (c, k, sk)
```

Maul(c, Δ)

```
c̃ ← c ⊕ Δ
Δ′ ← Δ ≫ 1
return (c̃, Δ′)
```

Decaps-Maul(c) Test (IND-CCA2 violations)

```
sus ← [ ];
Δ ← 1||0||...||0
(c, k, sk) ← GenInput()
repeat 8 · bytelen(c) times
    c̃, Δ′ ← Maul(c, Δ)
    try
        k̃ ← Decaps_sk(c̃)
        assert(k ≠ k̃)
    catch
        sus.append((c, Δ))
    Δ ← Δ′
return sus
```

GenInput()

```
(sk, pk) ← Keygen()
(c, k) ← Encaps_pk()
return (c, k, sk)
```

Maul(c, Δ)

```
c̃ ← c ⊕ Δ
Δ′ ← Δ ≫ 1
return (c̃, Δ′)
```

Decaps-Maul(c) Test (IND-CCA2 violations)

```
sus ← [ ];
Δ ← 1||0||...||0
(c, k, sk) ← GenInput()
repeat 8 · bytelen(c) times
    c̃, Δ′ ← Maul(c, Δ)
    try
        k̃ ← Decaps_sk(c̃)
        assert(k ≠ k̃)
    catch
        sus.append((c, Δ))
    Δ ← Δ′
return sus
```

GenInput()

```
(sk, pk) ← Keygen()
(c, k) ← Encaps_pk()
return (c, k, sk)
```

Maul(c, Δ)

```
c̃ ← c ⊕ Δ
Δ′ ← Δ ≫ 1
return (c̃, Δ′)
```

Decaps-Maul(c) Test (IND-CCA2 violations)

```
sus ← [ ];
Δ ← 1||0||...||0
(c, k, sk) ← GenInput()
repeat 8 · bytelen(c) times
    c̃, Δ′ ← Maul(c, Δ)
    try
        k̃ ← Decaps_sk(c̃)
        assert(k ≠ k̃)
    catch
        sus.append((c, Δ))
    Δ ← Δ′
return sus
```

Run in a fuzzer!

GenInput()

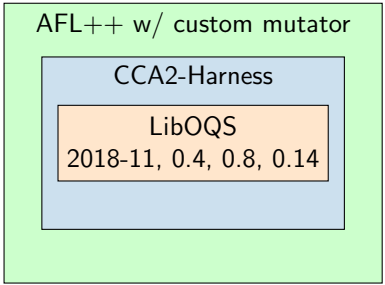
```
(sk, pk) ← Keygen()
(c, k) ← Encaps_pk()
return (c, k, sk)
```

Maul(c, Δ)

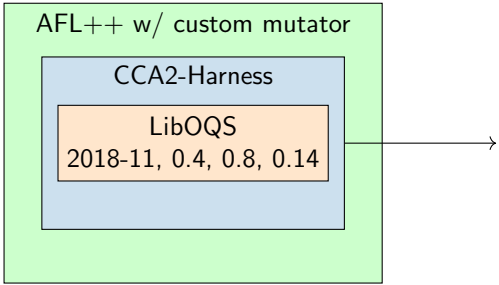
```
c̃ ← c ⊕ Δ
Δ′ ← Δ ≫ 1
return (c̃, Δ′)
```

In practice. . .

In practice. . .



In practice. . .



CCA2-Harness(sk, c, Δ, k)

1 : $\widetilde{k} \leftarrow \mathsf{Decaps}_{\mathsf{sk}}(\mathsf{c} \oplus \Delta)$

2 : **if** Δ contributes **and** $\widetilde{k} = k$

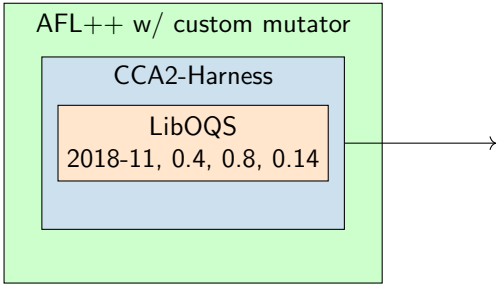
3 : `assert(0)`

4 : **if** Δ doesn't contribute **and** $\widetilde{k} \neq k$

5 : `assert(0)`

6 : **return** `SUCCESS`

In practice. . .



CCA2-Harness(sk, c, Δ, k)

1 : $\widetilde{k} \leftarrow \mathsf{Decaps}_{\mathsf{sk}}(\mathsf{c} \oplus \Delta)$

2 : **if** Δ contributes **and** $\widetilde{k} = k$

3 : `assert(0)`

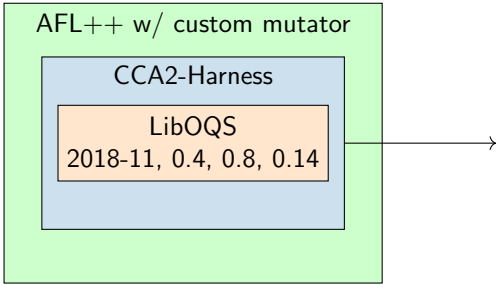
4 : **if** Δ doesn't contribute **and** $\widetilde{k} \neq k$

5 : `assert(0)`

6 : **return** `SUCCESS`

- Failing KEM implementations either don't satisfy IND-CCA2 security or buffer over-read.

In practice. . .



CCA2-Harness(sk, c, Δ, k)

1 : $\widetilde{k} \leftarrow \mathsf{Decaps}_{\mathsf{sk}}(\mathsf{c} \oplus \Delta)$

2 : **if** Δ contributes **and** $\widetilde{k} = k$

3 : `assert(0)`

4 : **if** Δ doesn't contribute **and** $\widetilde{k} \neq k$

5 : `assert(0)`

6 : **return** `SUCCESS`

- Failing KEM implementations either don't satisfy IND-CCA2 security or buffer over-read.
- Before looking at results, let's run a quick poll.

Pop quiz

Consider an IND-CCA2 secure KEM with a **correct** implementation.

Pop quiz

Consider an IND-CCA2 secure KEM with a **correct** implementation.

$$\boxed{\text{Keygen}} \longrightarrow (\mathsf{sk}, \mathsf{pk})$$

Pop quiz

Consider an IND-CCA2 secure KEM with a **correct** implementation.

$$\boxed{\text{Keygen}} \longrightarrow (\mathsf{sk}, \mathsf{pk})$$

$$\mathsf{pk} \longrightarrow \boxed{\text{Encaps}} \longrightarrow (\mathsf{c}, \mathsf{k})$$

Pop quiz

Consider an IND-CCA2 secure KEM with a **correct** implementation.

$$\boxed{\text{Keygen}} \longrightarrow (\mathsf{sk}, \mathsf{pk})$$

$$\mathsf{pk} \longrightarrow \boxed{\text{Encaps}} \longrightarrow (\mathsf{c}, \mathsf{k})$$

$$\widetilde{\mathsf{sk}} = \mathsf{sk} \oplus \boxed{0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 1 \mid 0}$$

Pop quiz

Consider an IND-CCA2 secure KEM with a **correct** implementation.

$$\boxed{\text{Keygen}} \longrightarrow (\mathsf{sk}, \mathsf{pk})$$

$$\mathsf{pk} \longrightarrow \boxed{\text{Encaps}} \longrightarrow (\mathsf{c}, \mathsf{k})$$

$$\widetilde{\mathsf{sk}} = \mathsf{sk} \oplus \boxed{0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 1 \mid 0}$$

$$(\widetilde{\mathsf{sk}}, \mathsf{c}) \longrightarrow \boxed{\text{Decaps}} \longrightarrow \widetilde{\mathsf{k}}$$

Pop quiz

Consider an IND-CCA2 secure KEM with a **correct** implementation.

$$\boxed{\text{Keygen}} \longrightarrow (\mathsf{sk}, \mathsf{pk})$$

$$\mathsf{pk} \longrightarrow \boxed{\text{Encaps}} \longrightarrow (\mathsf{c}, \mathsf{k})$$

$$\widetilde{\mathsf{sk}} = \mathsf{sk} \oplus \boxed{0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 1 \mid 0}$$

$$(\widetilde{\mathsf{sk}}, \mathsf{c}) \longrightarrow \boxed{\text{Decaps}} \longrightarrow \widetilde{\mathsf{k}}$$

Raise your hand if you think

$$\boxed{k = \widetilde{k}}$$

Pop quiz

Consider an IND-CCA2 secure KEM with a **correct** implementation.

$$\boxed{\text{Keygen}} \longrightarrow (\mathsf{sk}, \mathsf{pk})$$

$$\mathsf{pk} \longrightarrow \boxed{\text{Encaps}} \longrightarrow (\mathsf{c}, \mathsf{k})$$

$$\widetilde{\mathsf{sk}} = \mathsf{sk} \oplus \boxed{0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 0 \mid 1 \mid 0}$$

$$(\widetilde{\mathsf{sk}}, \mathsf{c}) \longrightarrow \boxed{\text{Decaps}} \longrightarrow \widetilde{\mathsf{k}}$$

Raise your hand if you think

$$\boxed{k = \widetilde{k}} \qquad \boxed{k \neq \widetilde{k} \ \ (\text{incl. } \widetilde{k} = \perp)}$$

Unexpected features

- We did not stop at testing IND-CCA2 security by mauling c in $\mathsf{Decaps}_{\mathsf{sk}}$.

Unexpected features

- We did not stop at testing IND-CCA2 security by mauling c in $\mathsf{Decaps}_{\mathsf{sk}}$.
- In our tests, we found an unexpected (to us) feature of Kyber/ML-KEM.

Unexpected features

- We did not stop at testing IND-CCA2 security by mauling c in $\mathsf{Decaps}_{\mathsf{sk}}$.
- In our tests, we found an unexpected (to us) feature of Kyber/ML-KEM.
- ML-KEM is obtained using the "Fujisaki-Okamoto" transform **with implicit rejection**.

$\mathsf{Decaps}_{\mathsf{sk}}(\mathsf{c})$

```
(sk', γ) ← sk
k ← PKE.Dec_sk'(c)
⋮
if c was detected as manipulated
    return H(c||γ)
else
    return k
```

Unexpected features

- We did not stop at testing IND-CCA2 security by mauling c in $\text{Decaps}_{\text{sk}}$.
- In our tests, we found an unexpected (to us) feature of Kyber/ML-KEM.
- ML-KEM is obtained using the "Fujisaki-Okamoto" transform **with implicit rejection**.

$\text{Decaps}_{\text{sk}}(\text{c})$

```
(sk', γ) ← sk
k ← PKE.Dec_sk'(c)
⋮
if c was detected as manipulated
    return H(c||γ)
else
    return k
```

Unexpected features

- We did not stop at testing IND-CCA2 security by mauling c in $\mathsf{Decaps}_{\mathsf{sk}}$.
- In our tests, we found an unexpected (to us) feature of Kyber/ML-KEM.
- ML-KEM is obtained using the "Fujisaki-Okamoto" transform **with implicit rejection**.

$\mathsf{Decaps}_{\mathsf{sk}}(\mathsf{c})$

```
(sk', γ) ← sk
k ← PKE.Dec_sk'(c)
⋮
if c was detected as manipulated
    return H(c||γ)
else
    return k
```

Unexpected features

- We did not stop at testing IND-CCA2 security by mauling c in $\mathsf{Decaps}_{\mathsf{sk}}$.
- In our tests, we found an unexpected (to us) feature of Kyber/ML-KEM.
- ML-KEM is obtained using the "Fujisaki-Okamoto" transform **with implicit rejection**.

$\mathsf{Decaps}_{\mathsf{sk}}(\mathsf{c})$

```
(sk', γ) ← sk
k ← PKE.Dec_sk'(c)
⋮
if c was detected as manipulated
    return H(c||γ)
else
    return k
```

Unexpected features

- We did not stop at testing IND-CCA2 security by mauling c in $\mathsf{Decaps}_{\mathsf{sk}}$.
- In our tests, we found an unexpected (to us) feature of Kyber/ML-KEM.
- ML-KEM is obtained using the "Fujisaki-Okamoto" transform **with implicit rejection**.

$\mathsf{Decaps}_{\mathsf{sk}}(\mathsf{c})$

$(\mathsf{sk}', \gamma) \leftarrow \mathsf{sk}$

$k \leftarrow \mathsf{PKE.Dec}_{\mathsf{sk}'}(c)$

$\vdots$

if c was detected as manipulated

  return $H(c||\gamma)$

else

  return k

Independent of γ

Unexpected features

- We did not stop at testing IND-CCA2 security by mauling c in $\mathsf{Decaps}_{\mathsf{sk}}$.
- In our tests, we found an unexpected (to us) feature of Kyber/ML-KEM.
- ML-KEM is obtained using the "Fujisaki-Okamoto" transform **with implicit rejection**.

$\mathsf{Decaps}_{\mathsf{sk}}(\mathsf{c})$

$(\mathsf{sk}', \gamma) \leftarrow \mathsf{sk}$

$k \leftarrow \mathsf{PKE.Dec}_{\mathsf{sk}'}(c)$

$\vdots$

if c was detected as manipulated

  return $H(c||\gamma)$

else

  return k

Independent of γ

- There is no obvious negative consequence, but it is also not intuitive.

Experiments

Experiments

- The bulk of testing is quite fast, yet LibOQS contains some schemes with large outputs.
  - To exhaustively test all schemes, runtimes can be long

Experiments

- The bulk of testing is quite fast, yet LibOQS contains some schemes with large outputs.
  - To exhaustively test all schemes, runtimes can be long
- We run LibOQS tests across two machines:
  - Machine A: two Intel Xeon Gold 6138 CPUs and 376 GiB of RAM.
  - Machine B: four Intel Xeon Gold 6252 CPUs and 754 GiB of RAM.

Experiments

- The bulk of testing is quite fast, yet LibOQS contains some schemes with large outputs.
  - To exhaustively test all schemes, runtimes can be long
- We run LibOQS tests across two machines:
  - Machine A: two Intel Xeon Gold 6138 CPUs and 376 GiB of RAM.
  - Machine B: four Intel Xeon Gold 6252 CPUs and 754 GiB of RAM.

Library	Wall time	Machine (Cores)
LibOQS 0.14.0	$12h\ 40m$	B (75)
LibOQS 0.8.0	$14h\ 23m$	A (31)
LibOQS 0.4.0	$12h\ 43m$	A (31)
LibOQS 11-2018	$8h\ 59m$	A (31)

Experiments

- The bulk of testing is quite fast, yet LibOQS contains some schemes with large outputs.
  - To exhaustively test all schemes, runtimes can be long
- We run LibOQS tests across two machines:
  - Machine A: two Intel Xeon Gold 6138 CPUs and 376 GiB of RAM.
  - Machine B: four Intel Xeon Gold 6252 CPUs and 754 GiB of RAM.

Library	Wall time	Machine (Cores)
LibOQS 0.14.0	$12h\ 40m$	B (75)
LibOQS 0.8.0	$14h\ 23m$	A (31)
LibOQS 0.4.0	$12h\ 43m$	A (31)
LibOQS 11-2018	$8h\ 59m$	A (31)

- This is an artifact of testing so many schemes! For most individual schemes, testing is fast.

Results

Highlights

Results

Highlights

- Various SW bugs (segfaults, . . .)

Results

Highlights

- Various SW bugs (segfaults, . . .)
- Ciphertext malleability in NTRU-{RSS-701, HPS-2048-677, HPS-2048-509} due to last few bits of ciphertext not contributing to decapsulation (OQS 0.4)

Results

Highlights

- Various SW bugs (segfaults, . . .)
- Ciphertext malleability in NTRU-{RSS-701, HPS-2048-677, HPS-2048-509} due to last few bits of ciphertext not contributing to decapsulation (OQS 0.4)
- Crashes and hangs on mauled ciphertexts for Big Quake, LedaKEM and Lima (OQS 2018-11), resulting in a ciphertext-validity oracle

Results

Highlights

- Various SW bugs (segfaults, . . .)
- Ciphertext malleability in NTRU-{RSS-701, HPS-2048-677, HPS-2048-509} due to last few bits of ciphertext not contributing to decapsulation (OQS 0.4)
- Crashes and hangs on mauled ciphertexts for Big Quake, LedaKEM and Lima (OQS 2018-11), resulting in a ciphertext-validity oracle
- Avoidable signature malleability in Falcon, Picnic (OQS 0.4), SNOVA and CROSS (OQS 0.14) due to non-unique encodings, byte-unaligned buffers, C API / DSS syntax gap

Results

Highlights

- Various SW bugs (segfaults, . . .)
- Ciphertext malleability in NTRU-{RSS-701, HPS-2048-677, HPS-2048-509} due to last few bits of ciphertext not contributing to decapsulation (OQS 0.4)
- Crashes and hangs on mauled ciphertexts for Big Quake, LedaKEM and Lima (OQS 2018-11), resulting in a ciphertext-validity oracle
- Avoidable signature malleability in Falcon, Picnic (OQS 0.4), SNOVA and CROSS (OQS 0.14) due to non-unique encodings, byte-unaligned buffers, C API / DSS syntax gap
  - Not a bug! (NIST does not ask for strong unforgeability) But good to explicitly know

Results

Highlights

- Various SW bugs (segfaults, . . .)
- Ciphertext malleability in NTRU-{RSS-701, HPS-2048-677, HPS-2048-509} due to last few bits of ciphertext not contributing to decapsulation (OQS 0.4)
- Crashes and hangs on mauled ciphertexts for Big Quake, LedaKEM and Lima (OQS 2018-11), resulting in a ciphertext-validity oracle
- Avoidable signature malleability in Falcon, Picnic (OQS 0.4), SNOVA and CROSS (OQS 0.14) due to non-unique encodings, byte-unaligned buffers, C API / DSS syntax gap
  - Not a bug! (NIST does not ask for strong unforgeability) But good to explicitly know
- Counterexamples to strong conservative exclusive ownership (S-CEO) and strong universal exclusive ownership (S-UEO) for some DSSs

Results

Highlights

- Various SW bugs (segfaults, . . .)
- Ciphertext malleability in NTRU-{RSS-701, HPS-2048-677, HPS-2048-509} due to last few bits of ciphertext not contributing to decapsulation (OQS 0.4)
- Crashes and hangs on mauled ciphertexts for Big Quake, LedaKEM and Lima (OQS 2018-11), resulting in a ciphertext-validity oracle
- Avoidable signature malleability in Falcon, Picnic (OQS 0.4), SNOVA and CROSS (OQS 0.14) due to non-unique encodings, byte-unaligned buffers, C API / DSS syntax gap
  - Not a bug! (NIST does not ask for strong unforgeability) But good to explicitly know
- Counterexamples to strong conservative exclusive ownership (S-CEO) and strong universal exclusive ownership (S-UEO) for some DSSs

- Against a "baseline" fuzzing campaign used as control, our tests only missed *few* SW bugs
- Cryptographically-informed testing is complementary to traditional fuzzing

Takeaways

Takeaways

- Cryptographically-informed testing finds classes of bugs not found by KATs and fuzzing
  - And it is relatively simple to set up using a fuzzer

Takeaways

- Cryptographically-informed testing finds classes of bugs not found by KATs and fuzzing
  - And it is relatively simple to set up using a fuzzer
- Mathematical syntax $\neq$ software API
  - Bugs nest in those gaps (byte misalignment, return values...)

Takeaways

- Cryptographically-informed testing finds classes of bugs not found by KATs and fuzzing
  - And it is relatively simple to set up using a fuzzer
- Mathematical syntax $\neq$ software API
  - Bugs nest in those gaps (byte misalignment, return values...)
- Undocumented unexpected features today could become protocol bugs tomorrow, it's worth identifying them
  - See Jackson *et al.*, "Seems Legit", ACM CCS 2019

Takeaways

- Cryptographically-informed testing finds classes of bugs not found by KATs and fuzzing
  - And it is relatively simple to set up using a fuzzer
- Mathematical syntax $\neq$ software API
  - Bugs nest in those gaps (byte misalignment, return values...)
- Undocumented unexpected features today could become protocol bugs tomorrow, it's worth identifying them
  - See Jackson *et al.*, "Seems Legit", ACM CCS 2019
- The general approach can be expanded to more complex protocols and integrated into CI pipelines
  - TLS-Attacker (Somorovsky, ACM CCS 2016) enables similar tests on TLS implementations

Takeaways

- Cryptographically-informed testing finds classes of bugs not found by KATs and fuzzing
  - And it is relatively simple to set up using a fuzzer
- Mathematical syntax $\neq$ software API
  - Bugs nest in those gaps (byte misalignment, return values...)
- Undocumented unexpected features today could become protocol bugs tomorrow, it's worth identifying them
  - See Jackson *et al.*, "Seems Legit", ACM CCS 2019
- The general approach can be expanded to more complex protocols and integrated into CI pipelines
  - TLS-Attacker (Somorovsky, ACM CCS 2016) enables similar tests on TLS implementations

Thank you

Full paper: ia.cr/2024/1122, artefact: github.com/jangilcher/cryptoTesting